
Lecture 7 - Software Architectures Distributed Approaches

Lecture Information

Master's Degree	Digital Automation Engineering (D.M.270/04)
Curriculum	Digital Infrastructure
Course	Distributed IoT Software Architectures
Lecture Title	Software Architectures Distributed Approaches
Author	Prof. Marco Picone (marco.picone@unimore.it)
License	Creative Commons Attribution 4.0

Table of Contents

- 7.1 Distributed Software Architectures - A Definition
- 7.2 Distributed Software Architectures - Fallacies
 - 7.2.1 Fallacy 1 - The network is reliable
 - 7.2.2 Fallacy 2 - Latency is zero
 - 7.2.3 Fallacy 3 - Bandwidth is infinite
 - 7.2.3.1 Stamp Coupling
 - 7.2.4 Fallacy 4 - The network is secure
 - 7.2.5 Fallacy 5 - Topology doesn't change
 - 7.2.6 Fallacy 6 - There is one administrator
 - 7.2.7 Fallacy 7 - Transport cost is zero
 - 7.2.8 Fallacy 8 - The network is homogeneous
- 7.3 Additional Distributed Challenges
 - 7.3.1 Distributed Transactions
- 7.4 Service-based Architecture
 - 7.4.1 Topology Characteristics
 - 7.4.2 Topology Variants
 - 7.4.2 Service-based Architecture - Summary & Rating
- 7.5 Event-Driven Architecture

-
- 7.5.1 Event-Driven Architecture - Topology Characteristics
 - 7.5.2 Event-Driven Architecture - Broker Topology
 - 7.5.3 Event-Driven Architecture - Mediator Topology
 - 7.5.4 Event-Driven Architecture - Topologies Comparison Summary
 - 7.5.5 Event-Driven Architecture - Other Aspects
 - 7.5.6 Choosing Between Request-Based and Event-Based
 - 7.5.7 Event-Driven Architecture - Summary & Rating
 - 7.6 Serverless Architecture
 - 7.6.1 Serverless Architecture - Key Terms & Design Concepts
 - 7.6.2 Serverless Architecture - Benefits & Challenges
 - References

7.1 Distributed Software Architectures - A Definition

Distributed Software Architecture refers to the design and organization of software systems that are distributed across multiple computing nodes, which may be located in the same physical environment or spread across different geographical locations. This architectural style emphasizes the interactions and communication between distributed components, enabling them to work together to achieve common goals.

Several common styles exist for distributed software architectures. Key approaches include:

- **Service-based architecture** — system decomposed into coarse-grained services that expose functionality via well-defined interfaces.
- **Event-driven architecture** — components interact asynchronously by emitting and consuming events, improving decoupling and scalability.
- **Space-based architecture** — components coordinate through a distributed shared space (e.g., tuple space) to avoid bottlenecks.
- **Service-oriented architecture (SOA)** — integration of reusable services across an enterprise, often via standard contracts and middleware.
- **Microservices architecture** — many small, independently deployable services, each owning its data and lifecycle.

-
- **Serverless architecture** — function-as-a-service model with event-triggered execution and managed scaling.

Course coverage (levels of detail):

- **Service-based** — introduced and discussed.
 - **Event-driven** — introduced and discussed.
 - **Microservices** — **in-depth: dedicated lectures, hands-on sessions, and laboratory work.**
 - **Serverless** — introduced and discussed.
-

7.2 Distributed Software Architectures - Fallacies

Distributed architectures provide superior **performance**, **scalability**, and **availability** compared to monoliths — but they introduce **important trade-offs** summarized by the “**Eight Fallacies of Distributed Computing**” (L. Peter Deutsch et al., Sun Microsystems, 1994).

A **fallacy** is an assumption believed to be true but not reliable in practice. The eight fallacies are:

- **The network is reliable** — networks fail; design for retries, timeouts, and graceful degradation.
- **Latency is zero** — communication incurs delay; prefer asynchronous patterns, batching, and locality.
- **Bandwidth is infinite** — throughput is limited; minimize payloads, compress, and avoid chatty protocols.
- **The network is secure** — assume it is untrusted; enforce authentication, authorization, and encryption.
- **Topology doesn't change** — nodes and routes can change; support discovery and dynamic configuration.
- **There is one administrator** — systems span multiple administrative domains; handle differing policies and access controls.

-
- **Transport cost is zero** — network usage has latency and monetary cost; reduce unnecessary transfers and optimize transfers.
 - **The network is homogeneous** — environments differ in hardware, OS, and middleware; design for interoperability and portability.
-

7.2.1 Fallacy 1 - The network is reliable

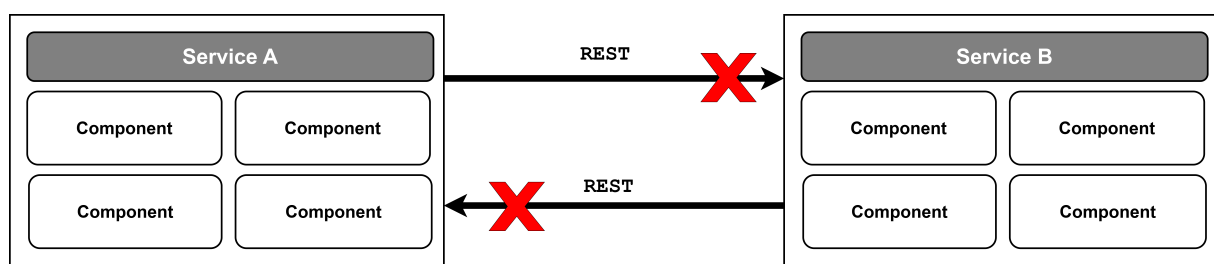


Figure 7.1: Schematic example of the unreliability of networks.

Developers often assume the **network is reliable** — but it isn't. **Design distributed systems assuming transient and partial failures.**

- **Reality:** networks experience drops, delays, partitions, DNS failures and intermittent outages.
- **Impact:** any architecture that relies on remote communication can fail if the network misbehaves.
- **Mitigations:** use for example:
 - **timeouts:** set reasonable timeouts for operations to avoid hanging indefinitely.
 - **retries with backoff:** retry failed operations with exponential backoff to avoid overwhelming the network.
 - **circuit breakers:** stop trying to reach a failing service after repeated failures, allowing it time to recover.
 - **idempotent operations:** design operations that can be safely retried without unintended side effects.
 - **caching:** store responses to avoid repeated network calls.
 - **graceful degradation:** provide fallback options when services are unavailable.
- **Design principle:** minimize synchronous network dependencies, isolate failures (bulkheads), and improve observability to detect and recover quickly.
- **Trade-off:** the more a system depends on the network (e.g., many microservices), the larger its exposure to network faults — design for resilience accordingly.

7.2.2 Fallacy 2 - Latency is zero

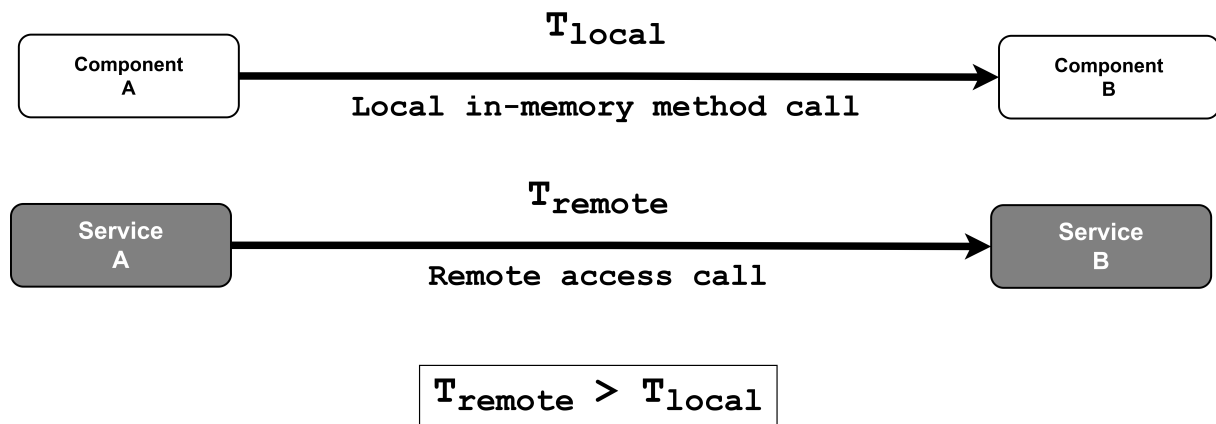


Figure 7.2: Example of local vs remote latency and their impact.

Developers often assume **latency is zero** in distributed systems — but it isn't. Relationships between components over a network introduce non-negligible delays and the following considerations apply:

- **Local vs remote latency**

- T_{local} (local method/function call) is typically in **nanoseconds (ns)** or **microseconds (μs)**.
- T_{remote} (REST, messaging, RPC) is typically in **milliseconds (ms)** and ****always greater than T_{local} ****.
- Conclusion: **latency is not zero** in distributed systems.

- **Why this matters**

- Architects must measure the **average round-trip latency** in the production environment (e.g., 60 ms vs 500 ms) to decide if a distributed design is feasible.
- Microservices amplify the issue because many fine-grained calls can accumulate latency.

- **Simple calculation**

- If average latency = **100 ms** per request, chaining **10** synchronous service calls adds **$\approx 1,000$ ms** to the user request.

- **Percentiles and the long tail**

- Average latency is useful, but **95th–99th percentiles** are critical. An average of **60 ms** with a **95th = 400 ms** will cause real-world performance problems.

-
- Design for the **long tail**: measure and mitigate high-percentile latencies.

- **Mitigations (brief)**

- Prefer **asynchronous calls**, **batching**, **caching**, **locality**, and **aggregation** to reduce synchronous call chains and exposure to network latency.
-

7.2.3 Fallacy 3 - Bandwidth is infinite

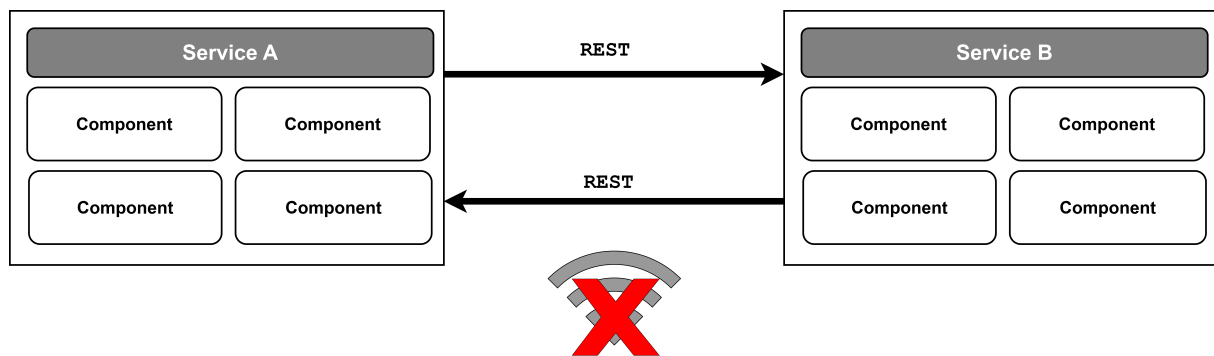


Figure 7.3: Example of bandwidth consumption in distributed systems.

Bandwidth is typically negligible inside a **monolith**, but in **distributed** architectures inter-service communication can consume significant bandwidth and degrade latency and reliability.

Example

- Service A: manages IoT devices.
- Service B: manages deployment/configuration.
- Service B returns 45 attributes $\approx$ **500 KB**, while Service A needs only the location $\approx$ **200 B** — an instance of **stamp coupling** (sending more data than required).
- At **2,000 requests/s**: $500 \text{ KB} \times 2,000 = 1,000,000 \text{ KB/s} = \mathbf{1,000 \text{ MB/s} \approx 1 \text{ GB/s}}$ ($\approx 8 \text{ Gbit/s}$) consumed just for that single interservice call.

Impacts

- Increased latency (*fallacy #2*), network congestion and cost, higher failure and reduced reliability (*fallacy #1*).
- **Stamp coupling** occurs when a service sends more data than the receiving service actually needs. This can lead to inefficient use of network resources, increased latency, and higher costs, especially in high-throughput systems.

Mitigations

- **Partial responses / field selection** (return only required fields).
- **Compression** or compact binary formats.
- **Caching** and co-location of services.
- **Aggregation / batching** and asynchronous messaging to reduce chatty calls.
- Clear data ownership and API design to avoid unnecessary transfers.

Design principle: assume bandwidth is limited — minimize payload size and chatty interactions (where and when possible).

7.2.3.1 Stamp Coupling

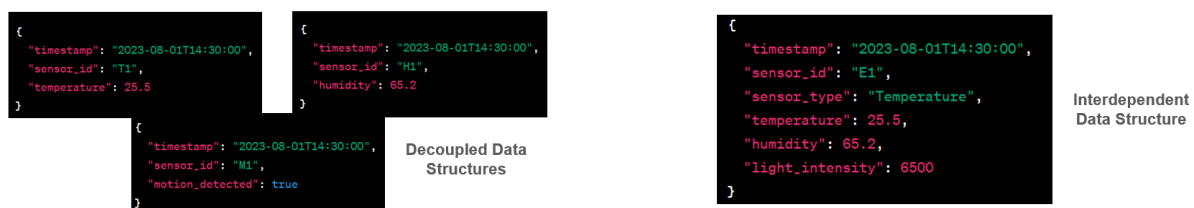


Figure 7.4: Simple example of stamp coupling and data structures.

Stamp coupling occurs when modules exchange large or composite data structures (records/objects) but each module uses only part of the data. This creates unnecessary dependencies and reduces modularity.

Key points:

- **Problem:** receiving modules become coupled to the structure and unused fields; changes to the shared structure can force widespread updates.
- **Consequences:** harder maintenance, larger payloads, increased bandwidth and risk of unintended side effects.
- **Mitigations:**
 - Pass only the **fields required** (field selection or small DTOs).

-
- Define **clear, minimal interfaces** or contracts.
 - Keep data structures **cohesive** and focused on a single concept.

Example:

- **Good (Left):** separate, self-contained structures per sensor type (temperature, humidity, light) — each module receives exactly what it needs.
- **Bad (Right):** one large structure containing all sensor fields — modules receive irrelevant data, increasing coupling and fragility.

Example: if Service B returns only the location (~200 B) instead of a full 500 KB record, at 2,000 req/s that saves roughly 400 KB/s of bandwidth.

Mitigations Examples - Private REST endpoints — expose minimal, purpose-specific endpoints for internal consumers.

- **Field selectors / partial responses** — allow callers to request only required fields (query params or Accept headers).
- **GraphQL / similar query layers** — let clients specify needed fields dynamically to avoid overfetching.
- **Consumer-Driven Contracts (CDCs)** — design APIs based on actual consumer needs to keep contracts small and stable.
- **Internal messaging / events** — publish compact messages (events) or use topics to avoid synchronous transfer of large objects.

Design principle: always transfer the minimal, well-defined data necessary; favor clear contracts and co-located logic to reduce bandwidth and coupling.

7.2.4 Fallacy 4 - The network is secure

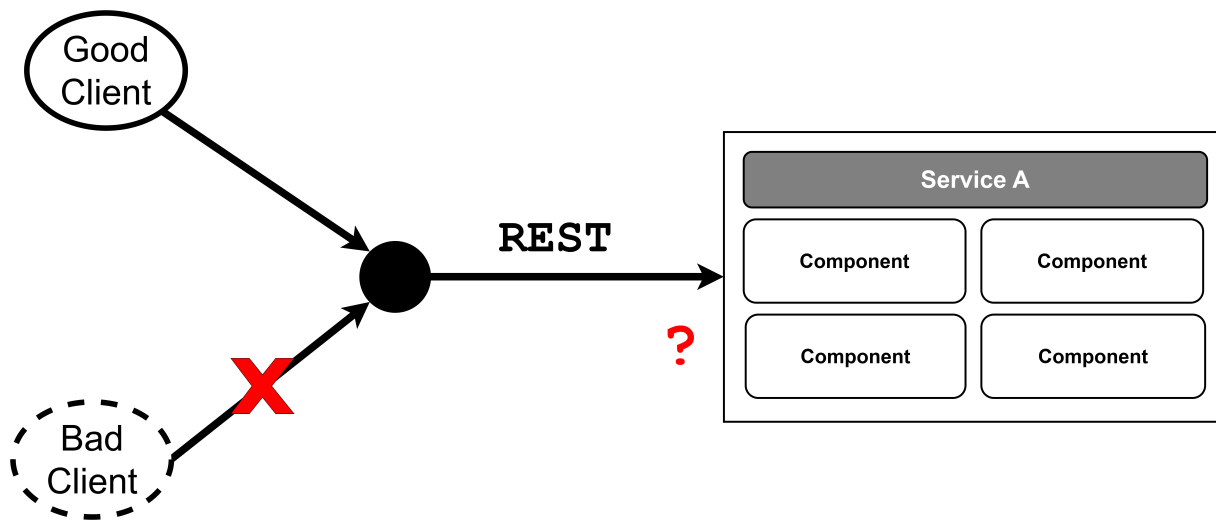


Figure 7.5: Example of the network is secure fallacy.

Most teams assume VPNs and firewalls make the network safe — but **the network is not secure**. In distributed architectures security becomes harder and must be designed explicitly.

- **Every endpoint must be protected:** each service instance, management API, and inter-service interface needs authentication, authorization, input validation and encryption.
- **Attack surface grows:** splitting a monolith into many services multiplies entry points and increases risk.
- **Interservice security matters:** internal traffic can carry threats; treat service-to-service calls as untrusted unless explicitly protected.
- **Operational cost and complexity:** securing all endpoints requires certificates, key management, policy enforcement and monitoring.
- **Performance trade-offs:** TLS, auth checks and additional middleware add latency, which helps explain why highly synchronous distributed designs can be slower.

Design guidance (brief): Adopt defense-in-depth and zero-trust principles by:

- minimizing exposed interfaces
- enforcing strong observability, and automating security configuration to ensure consistent policy enforcement and reduce human error.
- automating security configuration to ensure consistent policy enforcement and reduce human error.

7.2.5 Fallacy 5 - Topology doesn't change

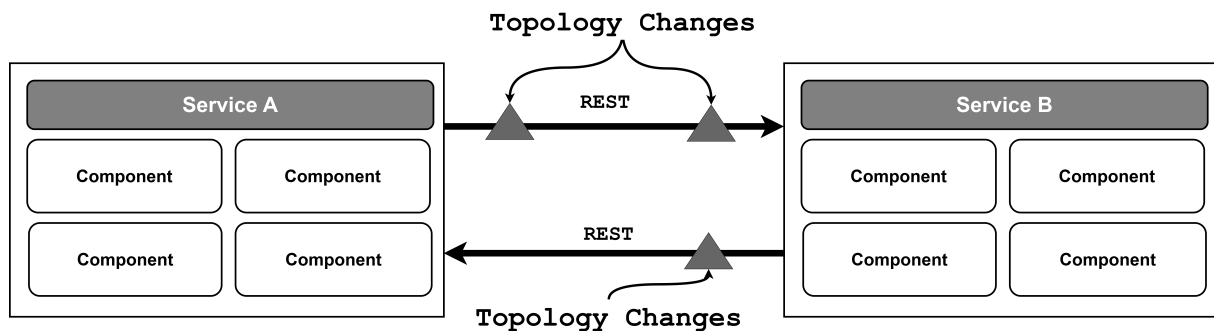


Figure 7.6: Topology changes in distributed systems.

Overall network topology includes all routers, switches, firewalls, hubs, links, VLANs, load-balancers and other appliances. Architects often assume this topology is fixed — but it changes frequently.

Key points:

- **Reality:** topology and network behavior change (configuration updates, firmware upgrades, routing changes, maintenance, failovers).
- **Impact:** seemingly minor infrastructure changes can invalidate **latency** and **bandwidth** assumptions, causing increased response times, timeouts, tripped **circuit breakers**, or unexpected request failures.
- **Examples:** MTU (Maximum Transmission Unit) or QoS (Quality of Service) changes, new fire-wall rules, route flaps, or a switch firmware update that adds milliseconds of latency or drops packets.

Mitigations / recommendations:

- Maintain continuous **communication** between architects, operations, and network admins (planned changes, maintenance windows).
- Use proactive **monitoring** and alerting for latency, packet loss, and topology changes (e.g., synthetic checks, traceroutes).
- Treat network as a dynamic dependency: design with conservative **timeouts**, retries with back-off, **bulkheads**, and graceful degradation.
- Include network change notifications in your **change management** and run performance tests after infra changes to validate assumptions.

Design principle: assume topology will change — build observability, coordination and resilience into the architecture to avoid surprises.

7.2.6 Fallacy 6 - There is one administrator

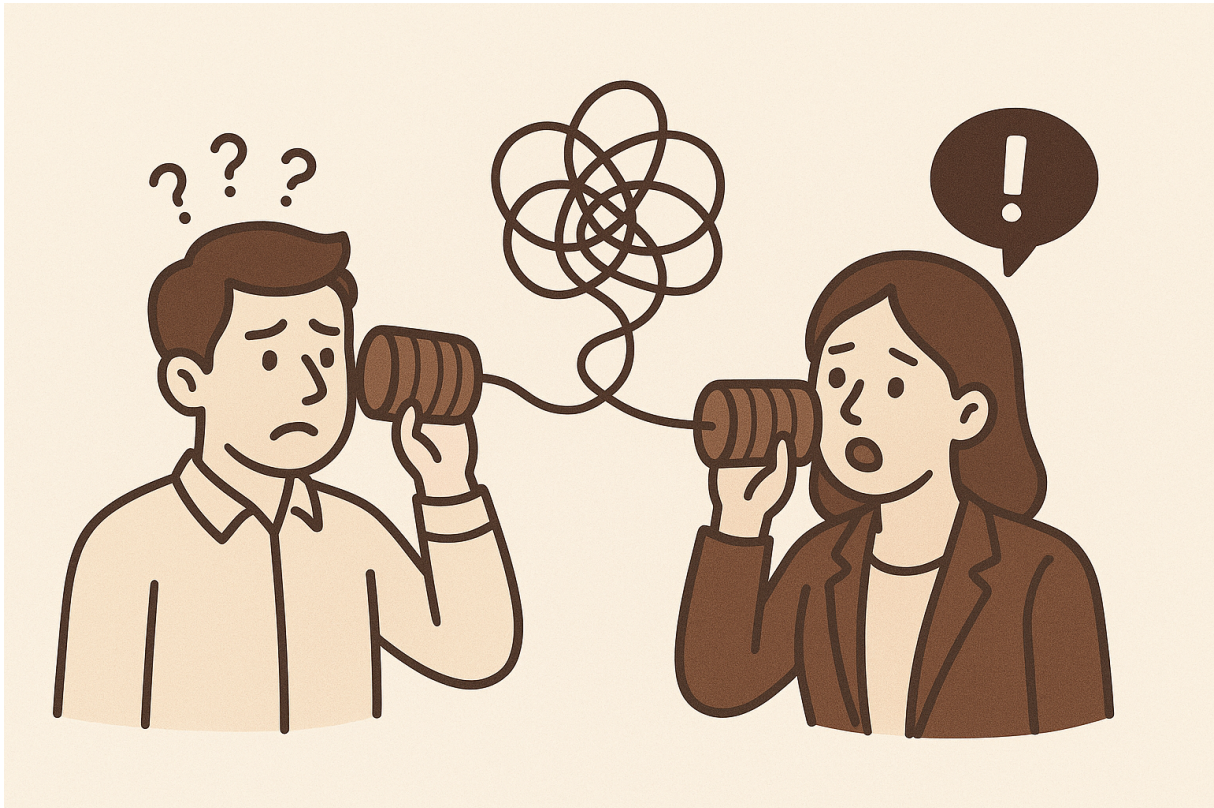


Figure 7.7: There is one administrator fallacy.

Large, distributed systems span multiple administrative domains (network, security, cloud, storage, site/network ops, platform teams), each with its own policies and change processes. The **assumption** of a single administrator **is flawed**.

Key points:

- **Reality:** many administrators and teams manage different parts of the infrastructure — you won't have a single person to contact for latency, routing, or configuration changes.
- **Impact:** assumptions about fast, centralized changes lead to surprises (unplanned topology or policy changes, differing SLAs - Service Level Agreements, delayed rollouts).
- **Consequences:** architects must plan for coordination overhead, scheduled change windows, and inconsistent policies across domains.

Practical mitigations:

- **Identify stakeholders** early (network, security, SRE, platform, NOC) and agree on communication and change procedures.

- **Formalize change management** (tickets, rollout plans, runbooks, SLAs) and include performance validation after infra changes.
- **Automate and codify** network/config management where possible (IaC, policy-as-code) to reduce human error and speed coordination.
- **Design for independence** (conservative timeouts, retries, bulkheads, graceful degradation) and add observability/synthetic tests to detect topology or admin-driven changes quickly.

Design principle: assume multiple administrators with differing policies — build coordination, observability, and resilience into your architecture to handle changes smoothly.

7.2.7 Fallacy 7 - Transport cost is zero

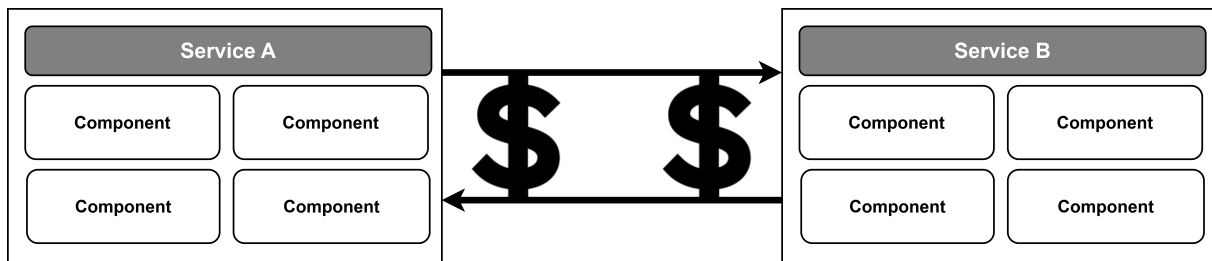


Figure 7.8: Transport cost is zero fallacy.

Transport cost here is the actual monetary and resource cost of moving data (not latency). Architects often confuse this with latency, but the fallacy concerns real expenses and infrastructure needs when you split a system.

- **Meaning:** network transfers, cross-zone traffic, cloud egress, additional middleware and appliances all carry direct and operational costs.
- **Why it matters:** distributed designs typically require more hardware and services — load balancers, gateways, proxies, firewalls, extra VMs/containers, new subnets, replication, and more monitoring/ops — which increases both capital and ongoing expenses.
- **Common hidden costs:** cloud egress fees, inter-AZ transfers, higher bandwidth plans, extra storage/replication, security and certificate management, and increased operational overhead (monitoring, backups, incident response).
- **Practical guidance:**
 - **Audit** current topology (capacity, bandwidth, security zones) and **estimate transfer/egress costs** before decomposing a monolith.
 - **Measure** expected traffic patterns and identify chatty interactions.

-
- **Design to reduce transfers:** partial responses/field selection, compression, caching, batching/aggregation, co-location, and event-driven or asynchronous patterns.
 - **Budget** for additional infra and operational work (networking, security, SLAs).
 - **Principle:** assume transport costs are non-zero — quantify and plan for them to avoid surprise bills and inadequate infrastructure.

Design principle: always consider the real costs of data transfer and infrastructure when designing distributed systems; optimize to minimize unnecessary expenses.

7.2.8 Fallacy 8 - The network is homogeneous

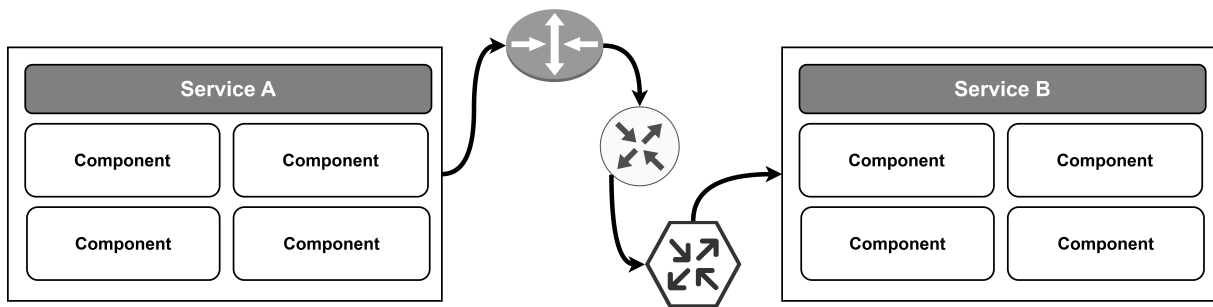


Figure 7.9: The network is homogeneous fallacy.

Assuming the **network is homogeneous** (built from a single hardware vendor) is **incorrect**. Most organizations run multi-vendor infrastructures — routers, switches, firewalls, wireless controllers and NICs (Network Interface Cards) from different vendors — so architects must design, test and operate for vendor heterogeneity, firmware differences and interoperability quirks.

- **Reality:** production networks are heterogeneous — multiple vendors, firmware versions, device models, links and configurations coexist.
- **Implication:** untested combinations and subtle vendor differences can cause interoperability issues (e.g., MTU mismatches, QoS quirks, firmware bugs) that lead to packet loss, jitter and throughput variance.
- **Effects:** this feeds directly into other fallacies — it undermines **reliability** (Fallacy #1), produces unexpected **latency** spikes and long-tail behavior (Fallacy #2), and reduces effective **bandwidth** or increases inefficiencies (Fallacy #3).

-
- **Practical mitigations:** test realistic vendor mixes, measure percentiles (not just averages), enforce conservative timeouts/retries and bulkheads, standardize/configure critical paths where possible, and design services to tolerate heterogeneity (observability, graceful degradation, and redundancy).

Design principle: assume heterogeneity, validate and design for its impacts rather than trusting a single, uniform network stack.

7.3 Additional Distributed Challenges

In addition to the eight fallacies, distributed systems introduce other practical challenges. Two key areas to remember:

- **Distributed logging**

- Problem: application and system logs are **scattered across many nodes**, in different locations and formats, making root-cause analysis and request tracing hard.
- Impact: correlating events, following a request flow, and diagnosing transient errors become time-consuming and error-prone.
- Mitigations (common practices):
 - ★ **Correlation IDs / request IDs** to link events across services.
 - ★ **Structured logs** (JSON) and consistent schemas for easier parsing and querying.
 - ★ **Centralized collection** (Prometheus, Splunk, Datadog, etc.) for aggregation, searching, and dashboards.
 - ★ **Distributed tracing** (OpenTelemetry, Jaeger) to visualize call graphs and latencies.
 - ★ **Sampling, retention policies, and access controls** to manage volume, cost, and privacy.
- Note: tooling helps but **doesn't eliminate** the complexity—design and operational practices are required.

- **Distributed transactions**

- Problem: maintaining ACID (Atomicity, Consistency, Isolation, Durability) semantics across multiple services and nodes is difficult or impractical.
- Common approaches: **sagas**, **two-phase commit (2PC)** where applicable, **idempotency**, and **compensation patterns** to handle partial failures.
- Design guidance: prefer **explicit eventual consistency**, design compensating actions, and make operations **observable and retryable**.

-
- See section 7.3.1 for an IoT transaction example and deeper discussion.

Design principle: distributed systems require careful design for logging and transactions; leverage patterns and tools but recognize inherent complexity. Keep these challenges in mind when decomposing systems: logging and transaction strategies are as important as service boundaries and network considerations.

7.3.1 Distributed Transactions

Distributed transaction (definition): a unit of work that must leave a system in a consistent state across multiple components — either all constituent operations succeed (commit) or the system recovers to a consistent state (rollback/compensation).

Smart-home IoT example (user leaving house — turn off lights, lock doors, set thermostat):

- **Begin transaction:** user triggers “leave home” action.
 - **Step 1 — Turn off lights:** send commands to light controllers; await acknowledgements.
 - **Step 2 — Lock doors:** send lock commands to door locks/windows; verify sensors/reporting.
 - **Step 3 — Set thermostat:** send setpoint to thermostat; confirm applied mode.
- **Commit:** if all steps succeed, mark the transaction complete and notify the user.
- **Failure handling:** if any step fails (e.g., a lock doesn’t engage), do one of:
 - **Rollback** where possible (e.g., revert lights/thermostat), or
 - **Compensating action / Saga** (e.g., alert user, retry with backoff, perform alternative safety steps), because true atomic rollback is often impractical across heterogeneous IoT devices.

Design considerations: prefer **idempotent commands**, timeouts, retries, clear **observable** states (ack/nack), user notifications, and compensating workflows to ensure safety and eventual consistency. In distributed IoT systems, treat transactions as coordinated workflows with explicit success/failure handling (compensation/sagas) rather than assuming instantaneous atomic commits.

7.4 Service-based Architecture

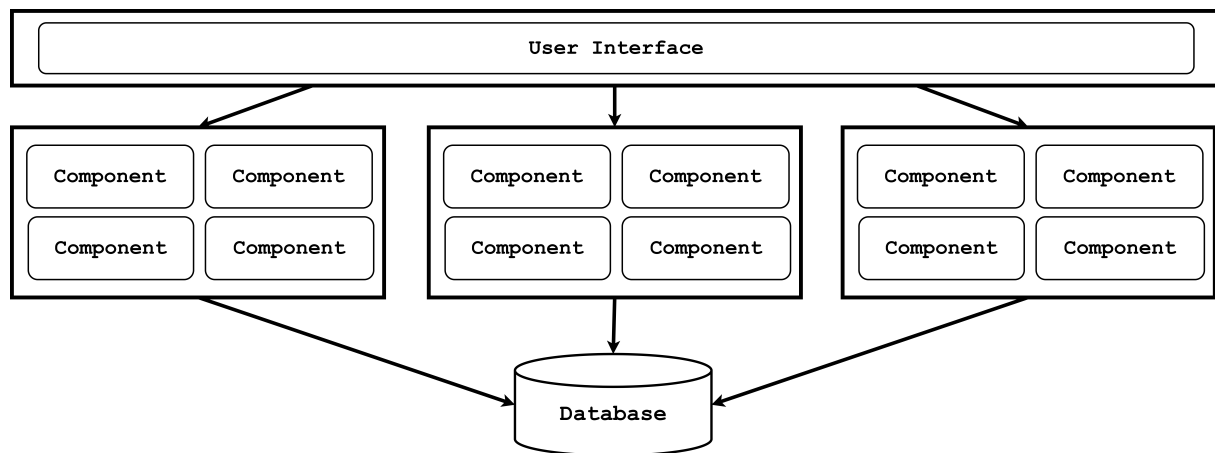


Figure 7.10: Schematic representation of Service-based Architecture.

Service-based Architecture (aka **Service-Oriented Architecture — SOA**) structures a system as a set of network-accessible, coarse-grained **services**. Coarse-grained means each service encapsulates a significant business capability (e.g., order management, user profile, inventory) rather than fine-grained functions. A coarse-grained service might handle all operations related to user management (registration, authentication, profile updates) rather than just a single function like “validate password”. A *service* is a self-contained module that encapsulates a business capability and can be developed, deployed and versioned independently.

Key points:

- **Granularity:** coarse-grained domain services (*larger than microservices*).
- **Independence:** services are modular and independently deployable (standalone apps or containers).
- **Access:** remote protocols (REST, RPC, messaging); often fronted by an API gateway.
- **Data ownership:** commonly uses a **shared/centralized database** (unlike pure microservices).
- **Trade-offs:** **pragmatic** and lower operational complexity/cost than microservices, yet still a distributed style — design for latency, failures and coupling.
- **When to use:** ideal for business applications that need **clear service boundaries** without the full overhead of microservices.

Summary: It is a distributed architecture style that structures an application as a collection of loosely coupled services. Each service encapsulates a specific business capability and communicates with other services over a network using well-defined interfaces and protocols.

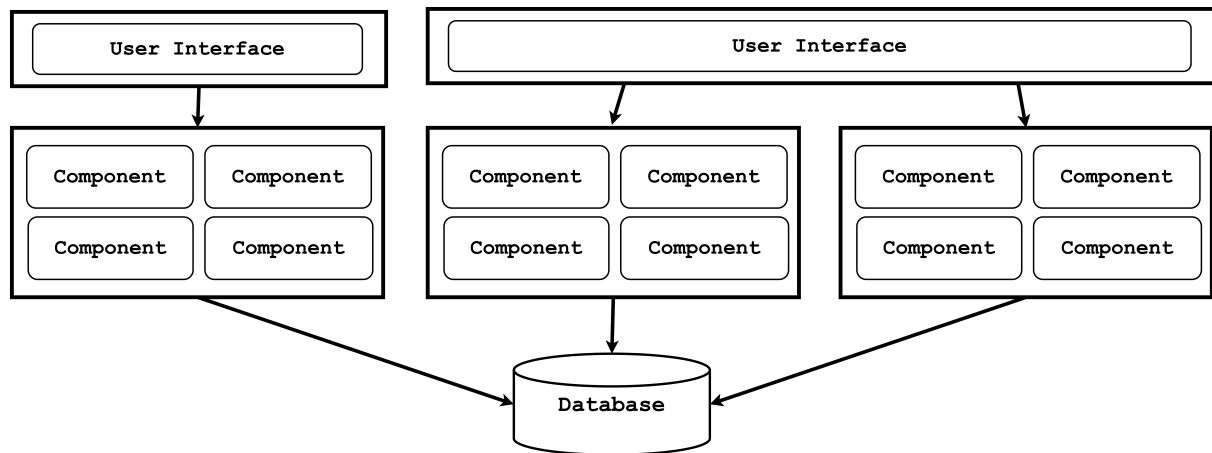
7.4.1 Topology Characteristics

Services within this architecture style are typically coarse-grained **portions of an application** (usually called **domain services**) that are independent and separately deployed. Main characteristics include:

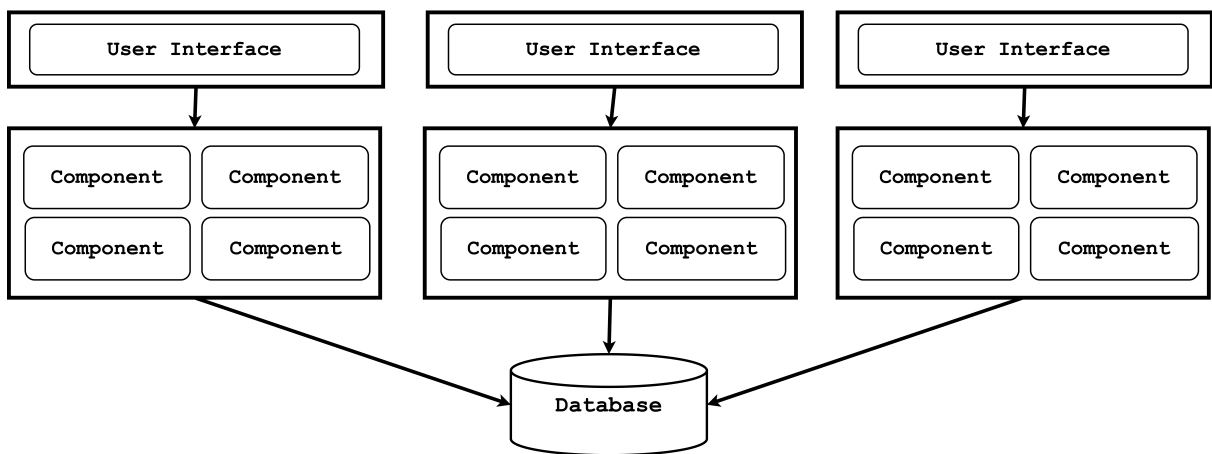
- **Macro layers:** separate **User Interface**, multiple remote **coarse-grained domain services**, and a shared **monolithic database**.
- **Service granularity & deployment:** services are coarse-grained domain modules, typically deployed as standalone applications/executables; containerization (e.g., Docker) is optional, not required.
- **Typical size:** small service count ($\approx 4\text{--}12$ services, commonly ~ 7) — fewer services reduces operational complexity but increases coupling.
- **Instances & scaling:** usually a single instance per domain service; horizontal scaling is possible (multiple instances) and requires **load balancing** to route UI traffic to healthy instances.
- **Access patterns:** services are called remotely from the UI using **REST** (most common), but **RPC, messaging, or SOAP** can be used. An **API gateway** or proxy is optional; many setups use a **service locator** to find services.
- **Shared data model:** a centrally **shared database** enables easy SQL joins and simple queries, but couples services tightly and makes schema changes harder.
- **Operational implications:** simpler than microservices (lower infra/ops), but watch for **database change management** and consider **database partitioning** or other strategies when evolving the schema or scaling data.
- **Design guidance:** favor minimal, well-defined service contracts, limit chatty interactions, and plan for controlled database evolution to balance simplicity and maintainability.

Design principle: Service-based Architecture offers a pragmatic distributed style with coarse-grained services and shared data, balancing modularity and operational simplicity. We will see that microservices architecture refines this style further by emphasizing independent data ownership and finer-grained services.

7.4.2 Topology Variants



Domain-Based User Interface



Service-Based User Interface

Figure 7.11: User Interface variations in Service-based Architecture.

Many topology variants exist within the service-based style. Common options:

- **Monolithic UI** — a single frontend calls multiple services (often via an API gateway); simple to operate but couples UI evolution.
- **Domain-specific UIs** — split the frontend by business domain so each UI aligns with its owning service; enables independent deployments and simpler UIs.

-
- **Backend-for-Frontend (BFF)** — per-client or per-domain backends that aggregate and tailor service responses, reducing chatty calls and payloads.
 - **Micro-frontends / federated UIs** — independently developed UI fragments composed at run-time; useful for large teams and isolated release cycles.
 - **Multiple client types** — specialized frontends (web, mobile, dashboards) may use dedicated UIs or share common APIs.

Design guidance: pick the variant that matches team boundaries, deployment independence, performance needs and acceptable operational complexity; always favor minimal coupling and clear API contracts.

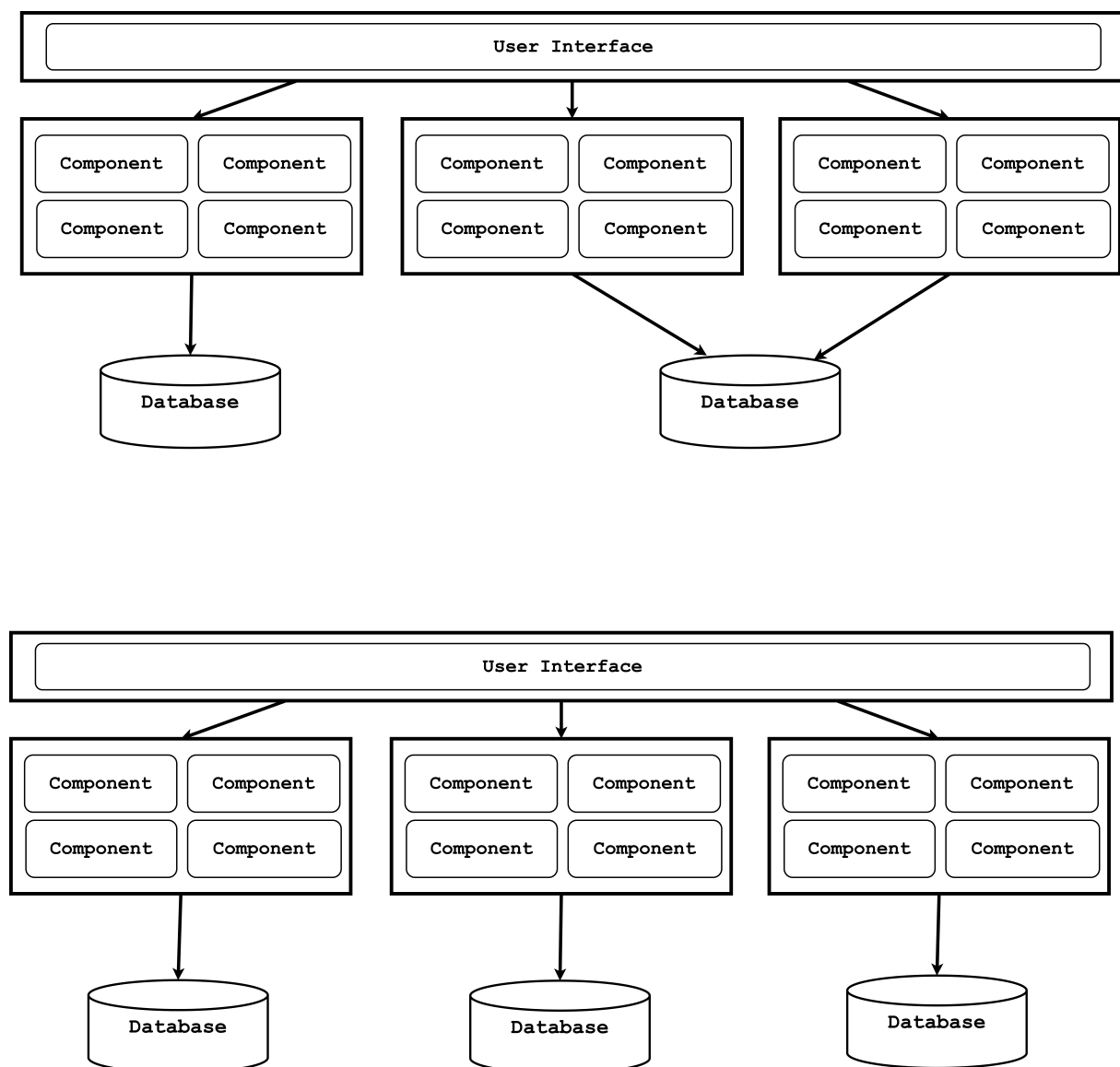


Figure 7.12: Storage variations in Service-based Architecture.

Similarly, split a monolithic database into multiple **domain-owned databases** (analogous to microservices). Key points:

- **Data ownership:** each database should contain only the data required by its domain; other services must not read another service's database directly.
- **Minimize coupling:** expose shared data via **service APIs** or **events** rather than direct DB access.
- **Avoid duplication and chatty calls:** when data must be shared, use controlled patterns (selective replication, consumer-driven contracts, or event streams).
- **Design trade-offs:** plan for **eventual consistency**, reconciliation mechanisms, and clear ownership to enable independent evolution and scaling.

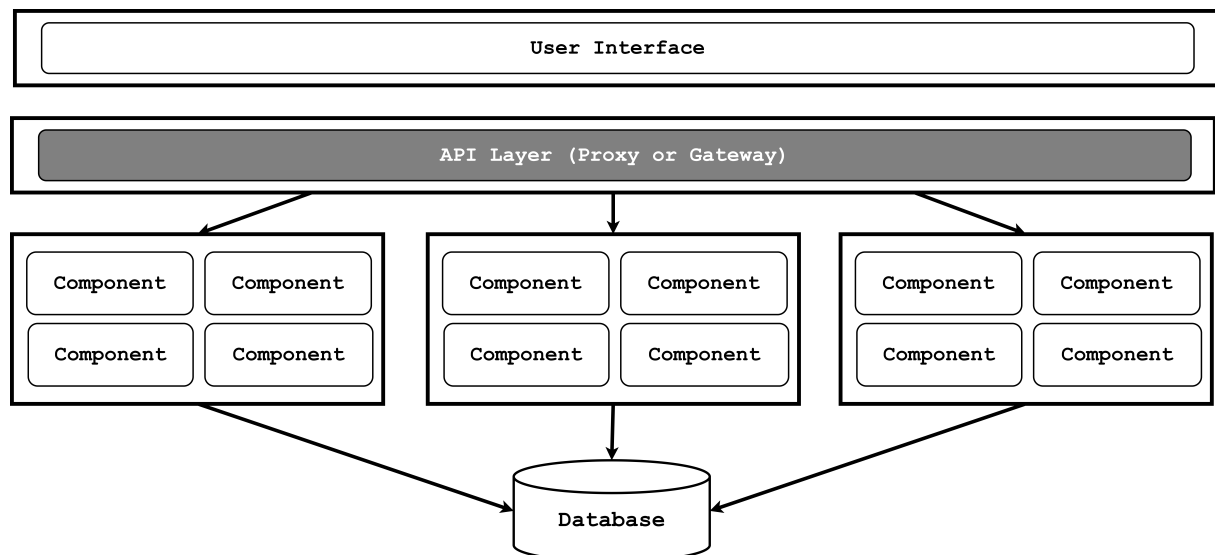


Figure 7.13: API Layer variation in Service-based Architecture.

Consider adding an **API layer** (reverse proxy / gateway) between the UI and domain services. It centralizes cross-cutting concerns and is especially useful when exposing services externally or consolidating shared functionality.

Key points: - **Use cases:** external APIs, multi-client aggregation (BFF), protocol translation, and enforcing organization-wide policies. - **Typical responsibilities:** routing, **auth/authz**, TLS termination, rate limiting, **metrics** and logging, auditing, request/response transformation, caching, circuit breaking, and service discovery. - **Benefits:** simplifies client code, reduces duplicated logic, enforces consistent security and observability, and enables central policy changes. - **Trade-offs:** introduces additional latency and operational scope—design for redundancy, scalability, and monitoring.

7.4.2 Service-based Architecture - Summary & Rating

Service-based Architecture — Summary & Rating

- **Focus:** domain-partitioned services that prioritize **domain logic** over presentation or persistence concerns.
- **Key benefits:**
 - **Agility:** faster feature delivery via independently deployable domain services.
 - **Testability:** smaller domain surface increases unit/integration test clarity.
 - **Deployability:** supports frequent, low-risk releases.
 - **Operational simplicity & cost:** less infrastructure/ops overhead than finer-grained patterns.
- **Resilience:** services are self-contained which limits blast radius; fewer cross-service dependencies reduce distributed-transaction complexity.
- **Scalability: moderate** — coarse granularity limits elasticity and resource efficiency compared to microservices; usually scales by increasing instances of whole services.
- **When to choose:** teams needing pragmatic modularization with lower operational cost and clearer domain boundaries, without the full operational burden of microservices.

Aspect	Effect	Recommendation
Agility	High — independent deployments speed changes	Use for fast iteration with clear domain boundaries
Testability	High — smaller scope per service	Keep services focused and well-scoped
Deployability	High — low-risk, frequent releases	Automate CI/CD for each service
Fault tolerance	Good — limited blast radius	Design services to be autonomous and idempotent
Scalability	Moderate — coarse granularity	Scale whole services or partition domains when needed
Complexity / Cost	Low → Moderate	Prefer when ops budget or team size is limited
Network & Transactions	Lower network chatter, fewer distributed transactions	Use APIs/events for explicit data sharing and consistency handling

7.5 Event-Driven Architecture

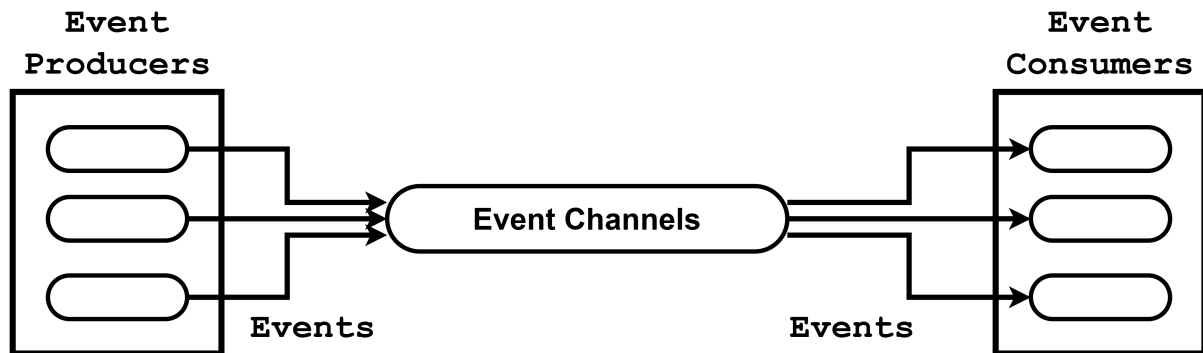


Figure 7.14: A schematic representation of an Event-Driven Architecture.

Event-Driven Architecture (EDA) defines system behavior around discrete **events**: occurrences (user actions, sensor readings, database changes, system notifications) that trigger processing and state changes.

Key characteristics:

- **Event sources:** producers of events (UIs, devices, services, DBs).
- **Event handlers / subscribers:** components registered to react to specific event types.
- **Asynchronous processing:** events are handled independently of the producer's execution flow, improving responsiveness and throughput.
- **Loose coupling:** components communicate via events rather than direct calls, reducing dependencies and allowing independent evolution.
- **Publish-subscribe model:** producers publish events; consumers subscribe to event streams or topics.
- **Message brokers / event buses:** optional intermediaries that route, buffer and persist events for reliability and scalability.
- **Scalability & resilience:** processing can be distributed and scaled horizontally; buffering helps absorb bursts and decouple failures.
- **Use cases:** real-time systems, IoT, streaming analytics, financial systems, and any domain requiring reactive, loosely coupled interactions.

EDA organizes systems around discrete **events** and asynchronous, **decoupled** processing. It is highly scalable and adaptable — suitable as a standalone style or embedded within other patterns (e.g., event-driven microservices). Use EDA when you need loose coupling, high throughput or reactive behavior; prefer request-based orchestration for simple, synchronous CRUD interactions or when strict transactional guarantees are essential.

7.5.1 Event-Driven Architecture - Topology Characteristics

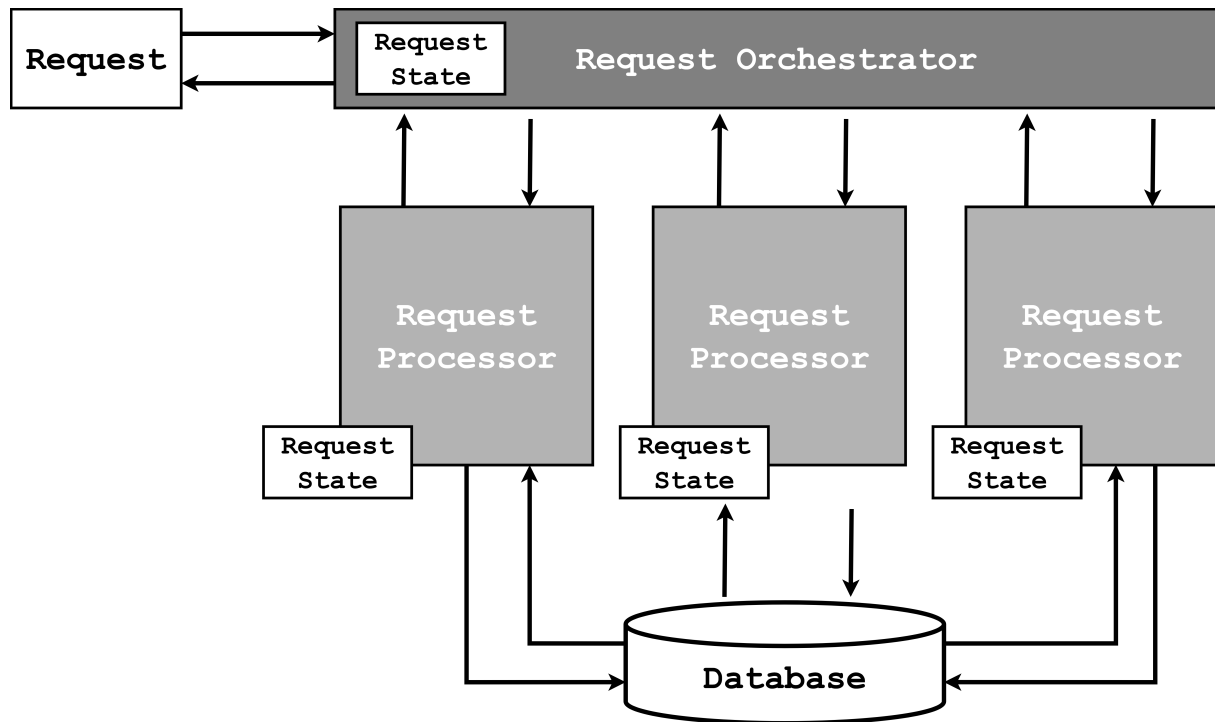


Figure 7.15: High-level view of Event-Driven Architecture components and interactions.

Two primary EDA topologies are commonly used:

- **Mediator topology**

- **Control / orchestration:** a centralized mediator (orchestrator) defines and manages the event workflow.
- **Best when:** you need predictable sequencing, complex business logic, or strict coordination between handlers.
- **Trade-offs:** simpler reasoning about flow but **tighter coupling** to the mediator and potential single-point complexity.

The Mediator topology centers control in a workflow or orchestration component that receives events or commands and decides the sequence of actions, enforces ordering and business rules, and tracks state progress. This approach simplifies reasoning about complex, multi-step business processes because a single, authoritative coordinator encodes the flow and can implement compensating actions, retries, and observability for each step. The trade-offs are increased coupling to the mediator, a larger blast radius if the mediator fails, and potential scaling challenges; these can be mitigated by using resilient workflow engines, persisting workflow state, partitioning orchestrations, or distributing mediators with leader election.

- **Broker topology**

- **Decoupling / responsiveness:** a broker (message bus) routes events to autonomous consumers; producers and consumers are loosely coupled.
- **Best when:** you need high throughput, elasticity, independent scaling, or dynamic consumer composition.
- **Trade-offs:** greater scalability and resilience but **less centralized control** and eventual consistency considerations.

The Broker topology relies on a message broker or event bus to route events between producers and consumers, favoring loose coupling and asynchronous processing. Producers publish events to topics or channels and consumers independently subscribe and react, which enables parallelism, elastic scaling of handlers, and natural fan-out for broadcast use cases. Because control is decentralized, the broker topology makes it easier to evolve components independently and to absorb bursts with durable queues, but it also shifts complexity to eventual consistency, message ordering semantics, delivery guarantees (at-least-once vs exactly-once), and the need for idempotent handlers. Operational concerns such as partitioning, consumer group coordination, retention policies, and monitoring are central to achieving predictable behavior at scale.

- **Design guidance**

- Use **Mediator** for workflow-centric scenarios where order and coordination matter.
- Use **Broker** for reactive, scalable systems where loose coupling and parallel processing matter.
- Hybrid approaches are common: orchestrate critical flows with a mediator and publish other events via a broker for async processing.

In practice, hybrid deployments are common: an orchestrator or mediator is used for flows that require explicit sequencing, transactional compensation, or strict business logic, while a broker is used for telemetry, integration, analytics, and other asynchronous fan-out needs. Schema management and contract governance matter in both models; using schema registries, versioning strategies, and lightweight contracts reduces coupling and prevents-breaking changes as consumers and producers evolve. Observability practices such as distributed tracing, correlation identifiers, dead-letter queues, and well-defined retry/backoff policies are essential to diagnose failures and to ensure safe retries and compensations across either topology.

Choosing between Mediator and Broker should be driven by control requirements, coupling tolerance, performance and scalability goals, and operational capacity. If predictable sequencing, centralized compensations, or auditability are primary concerns, a mediator or workflow engine is appropriate. If loose coupling, high throughput, elasticity, or broad event distribution are more important, a brokered, publish-subscribe architecture is preferable. When both needs exist, mix orchestration for crit-

ical workflows with brokered events for side effects and asynchronous processing, and design handlers to be idempotent, observable, and compatible with the chosen delivery guarantees.

7.5.2 Event-Driven Architecture - Broker Topology

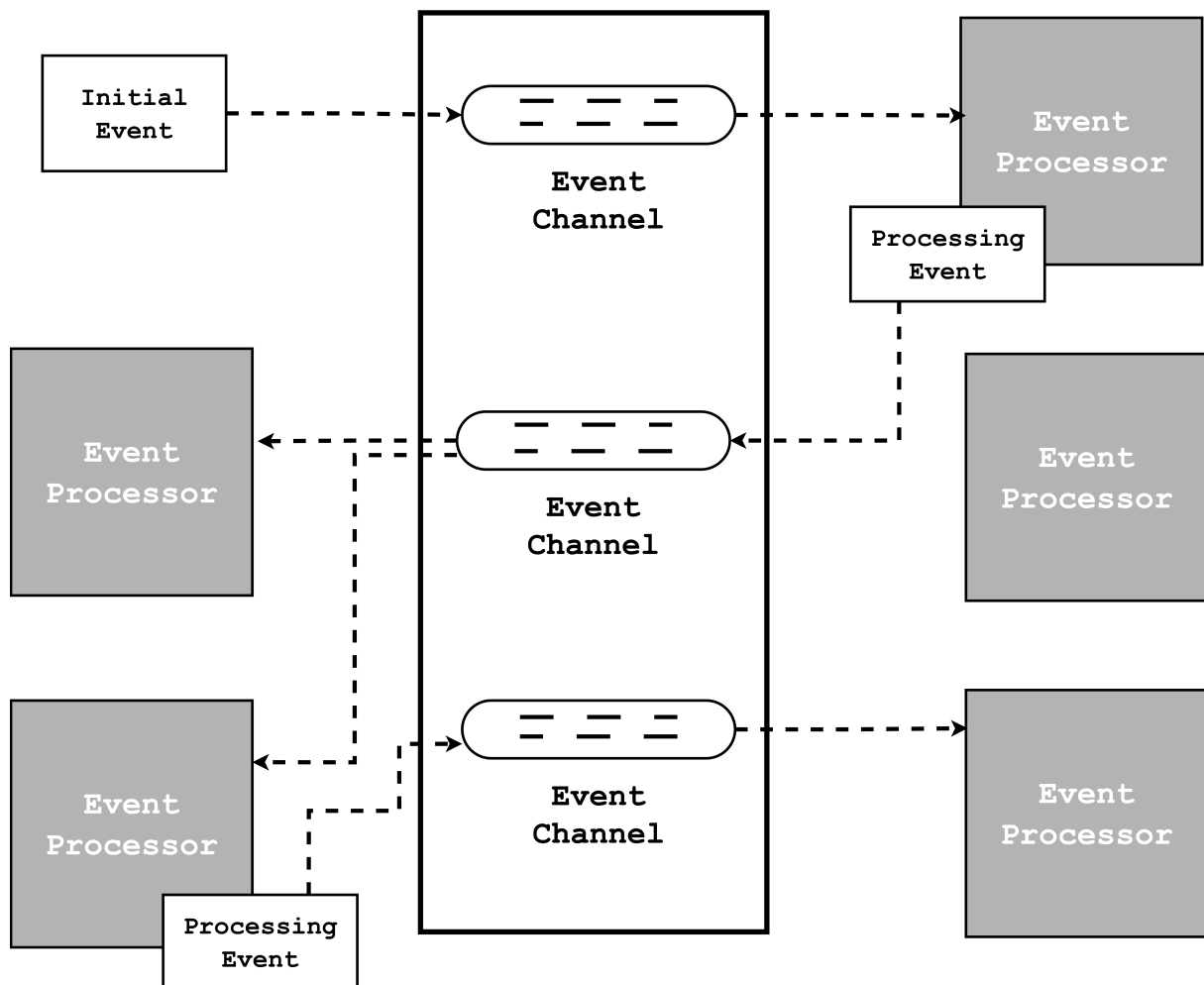


Figure 7.16: Broker Topology in Event-Driven Architecture.

The Broker topology routes events through a lightweight message broker instead of a central orchestrator. The broker handles distribution and buffering so producers and consumers remain loosely coupled.

Key points - **Decentralized routing:** producers publish events to the broker; the broker delivers them to subscribers (pub/sub / queue semantics). - **When to use:** simple or high-throughput event flows that don't require centralized sequencing or orchestration. - **Trade-offs to consider:**

message ordering, delivery guarantees (at-least-once vs exactly-once), idempotency of handlers, and dead-letter/retry policies. - **Examples of brokers:** RabbitMQ, ActiveMQ, Kafka, HornetQ.

Primary components - **Initiating event:** the original event produced by a source (UI, device, service). - **Event broker:** the message bus/queue that routes, buffers, and persists events. - **Event processor:** subscriber(s) that consume and handle events; can scale independently. - **Processing event:** resulting outputs or side effects (acknowledgements, new events, state changes).

Design Notes: prefer idempotent processors, clear schemas, and robust retry/monitoring (dead-letter queues, metrics, tracing) when using the broker topology.

The **initiating event** is published to an event channel on the **event broker**. In the broker topology there is no central mediator: an **event processor** subscribed to that channel consumes the initiating event and performs its task. When finished, the processor **asynchronously** emits a **processing event** to the broker describing the action taken. Other processors subscribed to that processing event may react, perform their tasks, and in turn publish further processing events. This asynchronous, decoupled chain continues until no processor subscribes to the latest event.

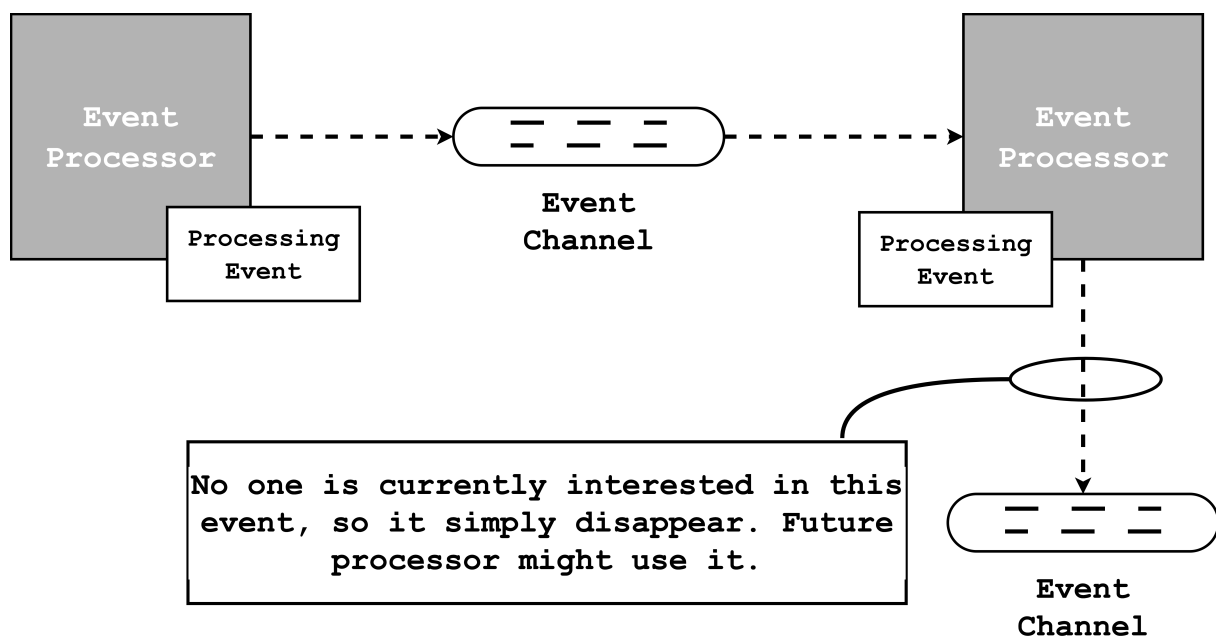


Figure 7.17: Additional Example of the Broker Topology with extensibility focus.

In the broker (pub/sub) topology, producers publish to **topics** and consumers independently subscribe. Best practice: each event processor should **publish a processing event describing the action it took**, even when no consumers currently listen.

Why do this:

- **Extensibility:** new consumers can be added later and immediately receive the needed data

without changing existing components.

- **Loose coupling:** producers and consumers remain independent; no direct integration work is required to add features.
- **Observability:** publishing actions creates an audit trail useful for debugging and monitoring.

Practical notes: - Use concise, well-versioned event schemas (topic names and payload fields) so future consumers can rely on stable contracts.

- Keep events idempotent or include identifiers so consumers can safely process retries or duplicates.
- Don't worry about occasional ignored messages—the small cost is outweighed by the agility and minimal integration effort when requirements evolve.

Example: a Notification processor sends an email and then publishes an “email.sent” event to a topic. If later you need to analyze sent emails, add an analyzer consumer that subscribes to that topic—no changes to the Notification processor required.

7.5.3 Event-Driven Architecture - Mediator Topology

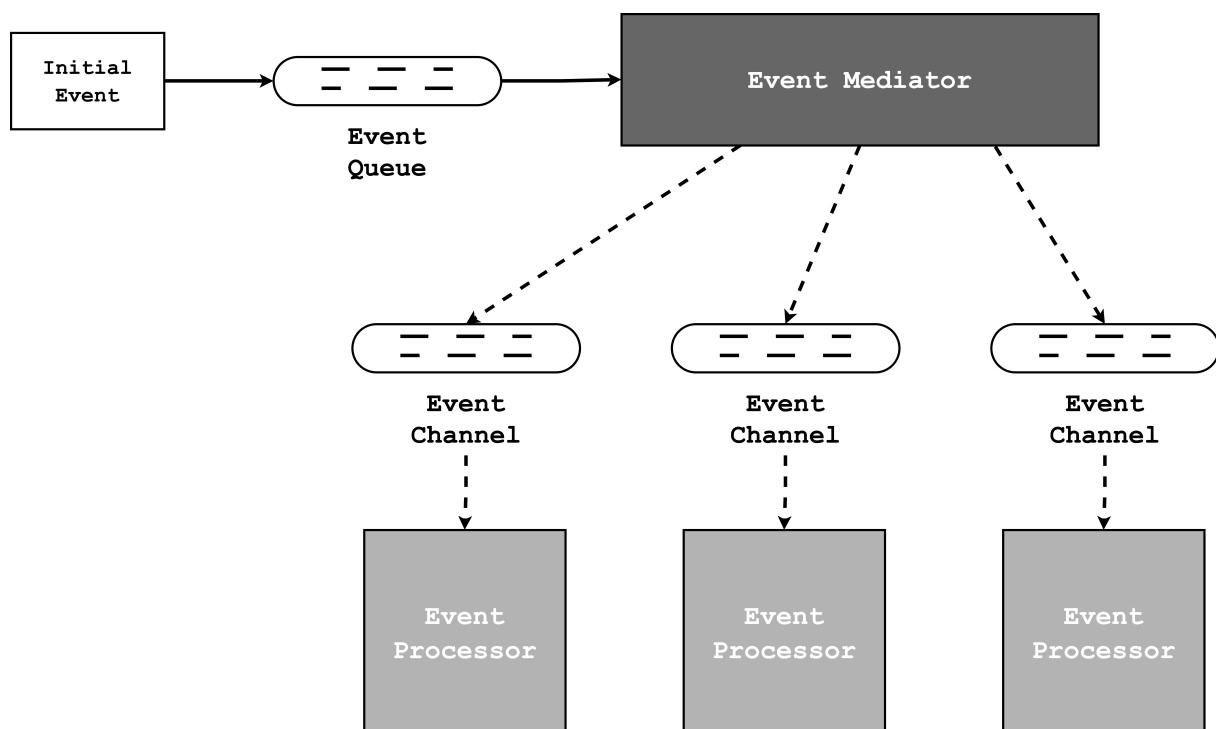


Figure 7.18: Mediator Topology in Event-Driven Architecture.

The mediator topology centralizes workflow control in an **event mediator** that coordinates initiating

events requiring ordered, multi-step processing. It is best for scenarios where sequencing, compensation, or strict coordination is needed.

Key components:

- **Initiating event** — the event that starts the workflow (e.g., user action, device message).
- **Event queue** — durable input queue that buffers initiating events for the mediator.
- **Event mediator** — the central orchestrator that encodes the workflow, invokes processors, tracks state, and issues compensations or retries.
- **Event channels** — point-to-point queues or channels used by the mediator to send tasks to specific processors.
- **Event processors** — workers that perform discrete steps and typically report completion back to the mediator.

Benefits and trade-offs:

- **Benefits:** deterministic sequencing, simpler compensation/retry logic, easier auditability and end-to-end visibility.
- **Trade-offs:** tighter coupling to the mediator, potential single-point complexity (mitigated by partitioning mediators by domain), and added operational responsibility.

Design guidance:

- Use mediator topology for complex, ordered business processes; combine with brokered events for fan-out/side effects when loose coupling is preferred.
- Ensure mediator state is durable and processors are idempotent to support safe retries and compensations.

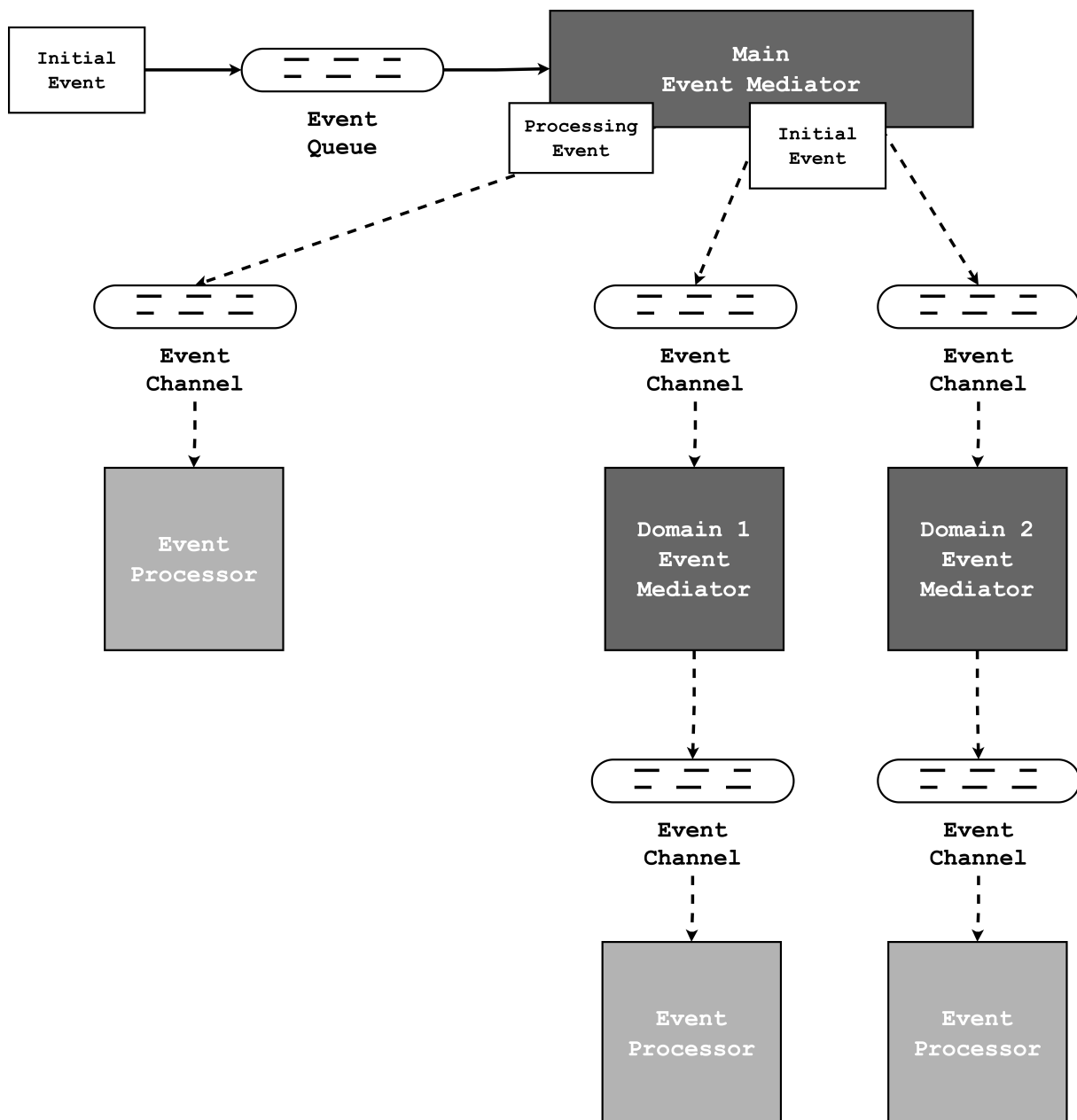


Figure 7.18: An example of multiple Event Mediator with different level of complexity and responsibilities within the same event-driven software architecture.

Initiating event: the message that starts the workflow. In the mediator topology it is placed on an *initiating event queue* and picked up by the **event mediator** (not broadcast as in the broker topology).

Flow overview: - The **event mediator** knows the workflow steps and emits corresponding **processing events** to dedicated **event channels** (typically point-to-point queues). - **Event processors** subscribe to those dedicated channels, perform work, and report completion back to the mediator. -

Unlike the broker topology, processors do **not** publish their results for all consumers; coordination and sequencing are controlled by the mediator.

Scalability and isolation: - Deploy multiple mediators (e.g., per domain or event group) to reduce single-point risk and increase throughput. Example domains: telemetry vs actuator control.

Design tips: - Keep mediator state durable, make processors idempotent, and model clear retry/compensation strategies to ensure safe recovery and observability. - Combine mediator orchestration for ordered flows with brokered events for fan-out side effects when appropriate.

7.5.4 Event-Driven Architecture - Topologies Comparison Summary

Broker Topology

Advantages	Disadvantages	Description
Highly decoupled event processors	Workflow control	Decentralized but harder to coordinate.
High scalability	Error handling	Scales well but errors are harder to manage.
High responsiveness	Recoverability	Fast response but recovery is complex.
High performance	Restart capabilities	Efficient but hard to restart processes.
High fault tolerance	Data inconsistency	Resilient but may cause inconsistent data.

Mediator Topology

Advantages	Disadvantages	Description
Workflow control	More coupling of event processors	Central control but more dependency.
Error handling	Lower scalability	Easier recovery but less scalable.

Advantages	Disadvantages	Description
Recoverability	Lower performance	Reliable but slower.
Restart capabilities	Lower fault tolerance	Easy to restart but less resilient.
Better data consistency	Modeling complex workflows	Consistent data but complex to design.

7.5.5 Event-Driven Architecture - Other Aspects

Other aspects to consider when designing an Event-Driven Architecture (in brief):

- **Error handling:** Detect, log and recover from processing failures using **retries with backoff**, circuit breakers, dead-letter queues and explicit compensation actions; make handlers **idempotent** to allow safe retries.
- **Preventing data loss:** Ensure durable delivery with **persistent event storage**, acknowledgements, replication and retention policies; use dead-letter queues and monitoring to surface lost or poisoned messages.
- **Broadcast capabilities:** Support fan-out via **pub/sub topics** so one event reaches multiple consumers; design for delivery semantics (at-least-once vs exactly-once), ordering constraints and consumer idempotency to avoid duplicates or inconsistency.
- **Request/Reply management:** Implement synchronous request-reply patterns over an async bus by using dedicated reply channels, timeouts, retries and clear correlation so callers receive timely and matched responses.
- **Correlation ID:** Attach a unique **correlation identifier** to related messages (requests, replies, and follow-up events) to enable tracing, correlate distributed work, and simplify debugging and observability.
- **Management queue:** Use a management/command queue to orchestrate request prioritization, rate-limiting and backpressure; combine with routing rules, retries and monitoring to balance load and prevent bottlenecks.

These features are complementary: design contracts, schemas and observability (tracing, metrics, alerts) up front to make them reliable and maintainable.

7.5.6 Choosing Between Request-Based and Event-Based

Prefer the request-based model when you need **synchronous control and certainty**; prefer the event-based model when you need **asynchrony, loose coupling and scale**. Key guidance:

- **Request-based (use when):**

- You require immediate responses and strict workflow control (e.g., CRUD, device profile retrieval).
- Transactions, strong consistency or simple error handling are important.
- Call chains are short and latency requirements are modest.

- **Event-based (use when):**

- You need high throughput, elasticity and reactive behavior (telemetry, analytics, notifications).
- Loose coupling, fan-out and resilience to partial failures are priorities.
- You can tolerate eventual consistency and design for idempotency and retries.

- **Trade-offs to weigh:**

- Request model → simpler debugging and testing, but can create tighter coupling and latency amplification.
- Event model → better scalability and extensibility, but increases complexity (ordering, retries, observability).

- **Hybrid recommendation:** mix both models—use request-based flows for synchronous control and event-based patterns for asynchronous, scalable side-effects and integrations to gain the benefits of each.

Choose based on workflow requirements (control vs. responsiveness), consistency needs, and operational capacity to manage complexity.

Advantages over

Request-Based	Trade-offs	Description
Better response to dynamic user content	Only supports eventual consistency	Adapts quickly but may have temporary data delays.
Better scalability and elasticity	Less control over processing flow	Grows easily but harder to manage flow.

Advantages over Request-Based		
Request-Based	Trade-offs	Description
Better agility and change management	Less certainty over outcome of event flow	Flexible but unpredictable results.
Better adaptability and extensibility	Difficult to test and debug	Easy to extend but harder to maintain.
Better responsiveness and performance	—	Fast reaction to events.
Better real-time decision making	—	Enables immediate insights and actions.
Better reaction to situational awareness	—	Quickly adapts to environment changes.

7.5.7 Event-Driven Architecture - Summary & Rating

- **Architecture nature:** Event-driven architectures are technically partitioned — domain behavior is distributed across multiple event processors, mediators, queues and topics rather than confined to a single service boundary.
- **Strengths**
 - **Performance:** High — gained from asynchronous communication and parallel processing.
 - **Scalability:** High — achieved via programmatic scaling of event processors (competing consumers).
 - **Fault tolerance:** High — decoupling, durable brokers and retries enable eventual processing and resilience.
 - **Evolutionary:** High — brokered events make it easy to add new consumers/features without changing producers.
- **Weaknesses**
 - **Complexity:** High — nondeterministic event flows can produce large, hard-to-reason about “event trees.”
 - **Testability & simplicity:** Low — many interleavings and scenarios make governance, end-to-end testing and debugging difficult.

-
- **Consistency & transactions:** Limited — favors eventual consistency; coordinating strong, cross-component transactions is harder.
 - **When to choose:** telemetry, real-time processing, high-throughput integrations, or systems requiring loose coupling and extensibility.
 - **When not to choose:** workflows demanding strict synchronous consistency, simple CRUD with tight transaction needs, or teams lacking maturity in observability and testing.
 - **Design guidance:** use broker topology for loose coupling and fan-out; use mediator/orchestration where ordering, compensation and auditability matter; invest in tracing, idempotency, schema governance and DLQs.
-

7.6 Serverless Architecture

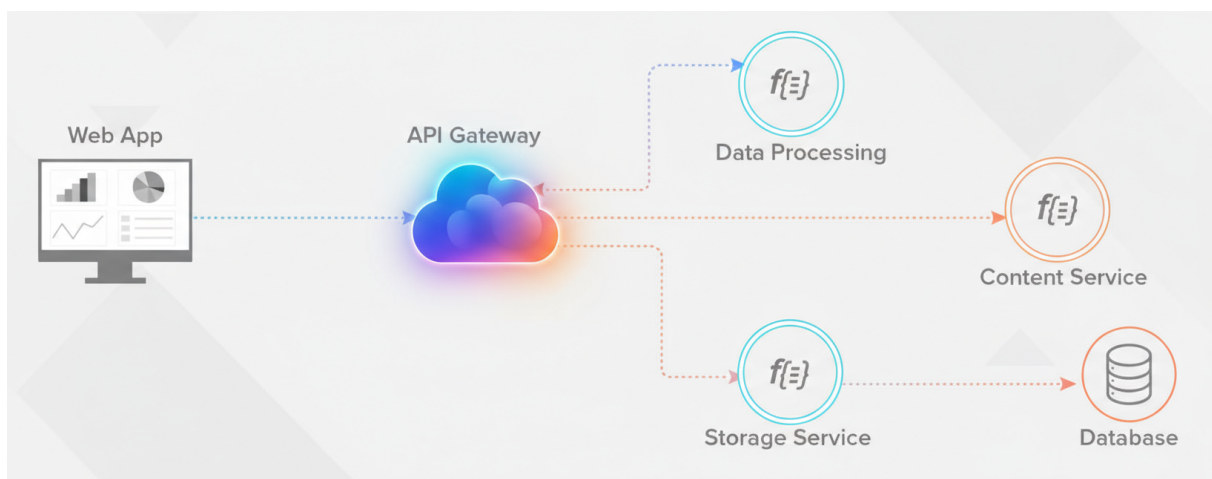


Figure 7.19: Simple schematic representation of Serverless Architecture.

Serverless architecture lets developers deploy and run application logic without managing servers or infrastructure. The cloud provider handles provisioning, OS/runtime management, scaling and capacity planning so you focus on code and events. The concept is often called **Function-as-a-Service (FaaS)** because discrete functions are deployed and executed in response to events. The Serverless idea is to abstract away infrastructure concerns so developers can build and deploy applications faster with lower operational overhead and it is related to the concepts of event-driven architecture since **functions are typically triggered by events**.

Key points: - **Provider-managed infrastructure:** servers, runtimes, and patching are handled by the cloud provider. - **Event-driven execution:** functions run in response to events (HTTP,

messaging, timers, storage). - **Automatic scaling:** compute scales up/down transparently to match demand. - **Pay-as-you-go billing:** you pay for actual execution time and resources used. - **Stateless, short-lived functions:** functions are typically ephemeral and should be idempotent. - **Built-in integrations:** easy connectivity to managed services (databases, queues, storage). - **Trade-offs:** potential cold starts, execution time/size limits, and increased vendor coupling.

Use serverless for event-centric, bursty workloads or to reduce operational overhead; evaluate limits and cost model before adoption.

7.6.1 Serverless Architecture - Key Terms & Design Concepts

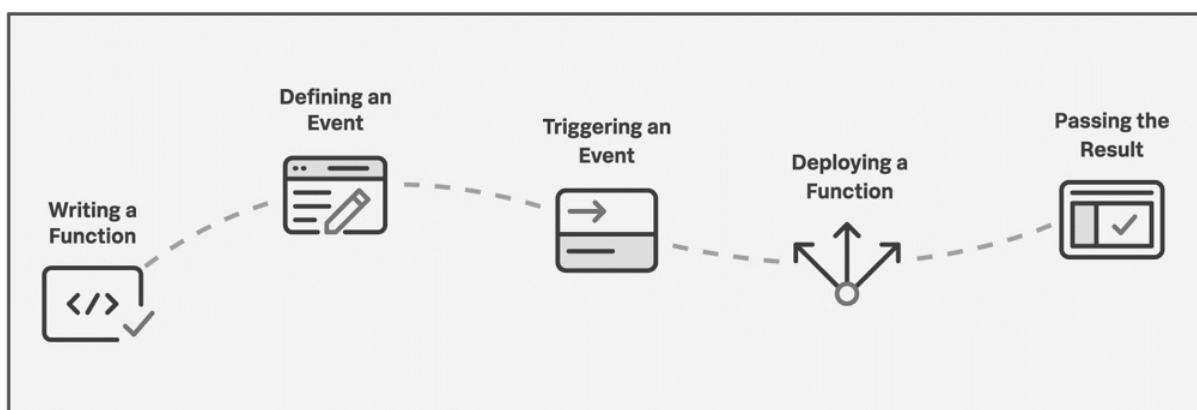


Figure 7.20: Design flow in Serverless Architecture.

- **Invocation:** a single execution of a function (one request or event → one run).
- **Duration:** wall-clock time the function runs (billing and performance metric).
- **Event:** the trigger for an invocation (HTTP, DB change, file upload, timer, external webhook, etc.). Serverless apps are event-driven.
- **Stateless:** functions do not retain in-memory state between invocations; state must be externalized (DB, cache, object store).
- **Cold start:** extra latency when a provider must provision a runtime/container before the first invocation after inactivity.
- **Warm execution:** faster subsequent invocations using an already-initialized runtime.

-
- **Concurrency limit:** max simultaneous function instances in a region (provider quota or configured limit).
 - **Orchestration:** coordination of multiple functions/workflows (step functions, workflow engines) to implement complex business processes.
 - **FaaS (Function-as-a-Service):** the serverless model where discrete functions are deployed and executed in response to events.

Simple FaaS workflow:

- Developers implement function code in a supported language (e.g., Python, Java, Go, JavaScript).
- Deploy the function to the cloud provider (packaging + configuration for triggers, memory, timeout).
- An **event** arrives and the provider invokes the function (reuse a warm runtime if available; otherwise incur a cold start).
- The function executes, returns a result or emits events, and the provider handles scaling, retries, and billing based on **duration** and concurrency.

Design tips (brief):

- Keep functions small and idempotent.
- Externalize durable state.
- Watch memory/time limits and concurrency to control cost and latency.
- Use orchestration for multi-step, stateful workflows instead of chaining synchronous function calls.

7.6.2 Serverless Architecture - Benefits & Challenges

Serverless adoption is rising — according to the 2022 State of Serverless report ([Link](#)), ~70% of AWS, ~60% of Google Cloud and ~49% of Azure customers use serverless solutions.

Key benefits - Reduced operational overhead: provider manages provisioning, scaling and runtime, letting teams focus on code.

- **Scalability:** automatic scaling to match workload spikes without manual intervention.
- **Cost efficiency:** pay-per-execution avoids charges for idle capacity (good for bursty/irregular loads).
- **Rapid development:** small, focused functions speed up implementation and deployment.
- **Granularity:** fine-grained functions improve modularity and maintainability.
- **Event-driven fit:** ideal for real-time, IoT, analytics and other event-centric use cases.

Main challenges - Vendor lock-in: platform-specific services and APIs make cloud migration costly.

- **Execution time limits:** short maximum runtimes constrain long-running tasks.
- **Debugging & observability:** tracing and local debugging are more complex and often need specialized tooling.
- **Cost trade-offs:** for steady, high-throughput workloads, serverless can be more expensive than reserved/managed infrastructure.

Use serverless when you need fast delivery, event-driven scale and low ops overhead; evaluate limits, observability needs and cost profile before committing.

References

- Fundamentals of Software Architecture: An Engineering Approach, by Mark Richards, Benjamin Lange, Neal Ford, Released February 2021, ISBN: 9781663728357
- Monolithic vs Microservices - [Link](#)
- Pattern: Microservice Architecture - [Link](#)
- Monolithic Architecture - [Link](#)
- Don't start with Monolith - [Link](#)
- Monolithic vs Microservice and all in between - [Link](#)
- Best Architecture for an MVP: Monolith, SOA, Microservices, or Serverless? - [Link](#)
- Microservices Introduction (Monolithic vs. Microservice Architecture) - [Link](#)
- How to break a Monolith into Microservices - [Link](#)
- Serverless Design Architecture: [Link](#)
- Serverless Architecture: [Link](#)
- A Guide to Serverless Architecture: [Link](#)